МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРИЙН УХААНЫ ТЭНХИМ

Пүрэвдоржийн Номин-Эрдэнэ

ҮЙЛДВЭРЛЭЛИЙН ДАДЛАГЫН ТАЙЛАН

Программ хангамж (D061302)

Цахим орчин дахь байршлын шинжилгээ

Улаанбаатар

2026 оны 02 сарын 5

МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРИЙН УХААНЫ ТЭНХИМ

ҮЙЛДВЭРЛЭЛИЙН ДАДЛАГЫН ТАЙЛАН

Программ хангамж (D061302)
Цахим орчин дахь байршлын шинжилгээ

Удирдагч: _____ Э.Цог-Эрдэнэ

Хамтран удирдагч: _____ К.Есенбек

Гүйцэтгэгч: _____ П.Номин-Эрдэнэ

Улаанбаатар
2026 оны 02 сарын 5

МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРИЙН УХААНЫ ТЭНХИМ

ПРОГРАММ ХАНГАМЖ АНГИЙН ОЮУТАН П.НОМИН-ЭРДЭНЭ ҮЙЛДВЭРИЙН ДАДЛАГЫН УДИРДАГЧИЙН ТОДОРХОЙЛОЛТ

2026 оны 1 сарын 29

2026 оны 01 сарын 08-ны өдрөөс 2026 оны 01 сарын 29-ний өдөр хүртэл 22B1NUM4640 кодтой оюутан П.Номин-Эрдэнэ нь UNIT LLC– Юу Эн Ай Ти ХХК байгууллагад үйлдвэрлэлийн дадлагыг амжилттай хийж гүйцэтгэсэн болно.

Оюутан ажлын байрны дүрэм журмыг чандлан сахиж, хугацаанд нь хариуцлагатай ажилласан бөгөөд дадлагын хугацаанд дараах ажлуудыг гүйцэтгэв:

- Сошиал медиагаас цугларч буй их хэмжээний өгөгдлийн бүтэц, онцлогийг ойлгож судлах
- Постын текстээс байршлын мэдээллийг ялган авах аргачлал судлах, боловсруулах
- Боловсруулсан өгөгдлийг өгөгдлийн санд хадгалах, dashboard хэлбэрээр харуулах web page хөгжүүлэх
- Monitoring системийн дотоод логик, өгөгдлийн урсгалыг ойлгох

Үнэлгээ: _____ (10 онооноос оноо)

Үйлдвэрлэлийн дадлагын хариуцсан удирдагч Э.Цог-Эрдэнэ : _____

МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
Программ хангамжийн ангийн IV түвшний оюутан П.Номин-Эрдэнэ

ДАДЛАГЫН АЖЛЫН ТӨЛӨВЛӨГӨӨ

Хүснэгт 1: Дадлагын ажлын төлөвлөгөө

№	**Гүйцэтгэх ажил**	**Хугацаа**	**Биелэлт**	**Удирдагчийн үнэлгээ**
1	Байгууллага, үндсэн систем, дотоод дүрэм журам,дадлагын хугацаанд гүйцэтгэх даалгавар зэрэгтэй танилцах.	01/08–01/09		
2	Текстээс байршилын мэдээллийг ялгаж авах аргачлал, ашиглаж болох технологи, хэрэгжүүлэлтийн аргуудыг судлах	01/12–01/13		
3	Судалсан аргачлалуудыг ашиглан excel дээрх өгөгдөл дээр боловсруулалт хийж, байршлын мэдээллийг ялгаж турших	01/14–01/16		
4	Үр дүнг удирдагчид шалгуулан дахин засаж сайжруулах	01/19–01/20		
5	Удирдагчтай хамтран өгөгдлийн санг зохиомжилж, боловсруулсан өгөгдлийг db-д нэмэх	01/21–01/23		
6	Хөгжүүлэлтийн орчинг тохируулах ашиглах технологиудыг судлах	01/26–01/27		
7	Хөгжүүлэлт, кодын хэрэгжүүлэлтийг гүйцэтгэх	01/27–01/29		
8	Дадлагын ажлын танилцуулга бэлтгэх, үр дүн нэгтгэх, тайлан бичиж дуусгах.	01/28–01/29		

Баталсан: Албан байгууллагын дадлага удирдагч Э.Цог-Эрдэнэ: _____

Төлөвлөгөө боловсруулсан: Оюутан П.Номин-Эрдэнэ:

Зохиогчийн баталгаа

Миний бие П.Номин-Эрдэнэ нь “ПРОГРАММ ХАНГАМЖ ХӨТӨЛБӨРИЙН ҮЙЛДВЭРЛЭЛИЙН ДАДЛАГЫН ТАЙЛАН” сэдэвт судалгааны ажлыг гүйцэтгэсэн бөгөөд дараах зүйлсийг баталж байна:

- Ажил нь бүрэн эхээрээ өөрийн бие даан гүйцэтгэсэн ажил болно.
- Эх сурвалж ашигласан бол зохих ёсоор иш татаж, ном зүйд оруулсан.
- Бусдын хийсэн ажил, судалгааг хуулбарлаагүй, ашигласан бол ишлэл, зүүлт хийсэн.
- Ажлын үр дүнг үнэн зөвөөр илэрхийлсэн бөгөөд миний хийсэн ажил, үзүүлсэн дэмжлэгийг дүгнэсэн байна.

Гарын үсэг: _____ Огноо: _____

Гарчиг

Удиртгал **1**

1 БАЙГУУЛЛАГЫН ТАНИЛЦУУЛГА **2**

1.1 UNIT LLC/Юу Эн Ай Ти ХХК танилцуулга 2

1.2 Sumbee.mn системийн зорилго 2

1.2.1 Social monitoring гэж юу вэ? 2

1.2.2 Sumbee-ийн үндсэн боломжууд 2

2 ИЖИЛ СИСТЕМҮҮДИЙН СУДАЛГАА **3**

2.1 Олон улсад 3

2.2 Монголын зах зээл дэх мониторингийн хэрэгслүүд 3

2.3 Sumbee.mn системийн давуу тал 3

3 СИСТЕМИЙН ШААРДЛАГА **5**

3.1 Системийн хэрэглэгч 5

3.2 Функциональ хэрэглэгчийн шаардлага 5

3.3 Функциональ бус шаардлага 6

4 АШИГЛАСАН ТЕХНОЛОГИУД **7**

4.1 Python програмчлалын хэл 7

4.1.1 Feature-д гүйцэтгэсэн үүрэг 7

4.2 Dictionary-based approach 7

4.2.1 Давуу тал 8

4.3 MSSQL өгөгдлийн сан 8

4.3.1 Яагаад MSSQL сонгосон бэ? 8

4.3.2 Feature-д гүйцэтгэсэн үүрэг 8

4.4 Backend хөгжүүлэлт – ASP.NET MVC 9

4.4.1 Архитектурын зохион байгуулалт 9

4.5 Frontend хөгжүүлэлт – ASP.NET MVC 9

4.5.1 Razor View ба өгөгдөл дамжуулалт 9

4.5.2 DataTables ашигласан интерфэйс 9

4.5.3 JavaScript ба UI логик 10

5 ХЭРЭГЖҮҮЛЭЛТ **11**
5.1 Байршлын dictionary үүсгэх . . . 11
5.2 Байршлын мэдээлэл ялгах алгоритм . . . 11
5.2.1 Байршлын нэршил илрүүлэх алгоритмын ажиллах зарчим . . . 11
5.2.2 Алгоритмын онцлог . . . 13
5.3 Өгөгдлийн санд хадгалах . . . 13
5.4 Хөгжүүлэлтийн орчин тохируулах . . . 14
5.4.1 Хөгжүүлэлтийн орчин суулгах . . . 14
5.4.2 Төслийн эх код татах . . . 14
5.4.3 Dependencies ба өгөгдлийн сангийн холболт . . . 14
5.4.4 Төслийг local орчинд ажиллуулах . . . 14
5.5 Вэб хөгжүүлэлт . . . 14
5.5.1 Backend логик (ASP.NET MVC) . . . 15
5.6 Туршилт ба баталгаажуулалт . . . 16

Дүгнэлт **17**

Ном зүй **18**

A ХАВСРАЛТ Кодын хэсэг **19**

Зургийн жагсаалт

5.1 Өгөгдлийн сангийн бүтэц . . . 13
5.2 Газрын зураг,статистик бүхий dashboard . . . 15
5.3 Байршлын мэдээлэл илэрсэн постуудын жагсаалт . . . 16
5.4 Бүртгэлийг засаж, өөрчлөх боломжтой цонх . . . 16

Хүснэгтийн жагсаалт

1 Дадлагын ажлын төлөвлөгөө . . . 3

Кодын жагсаалт

A.1 Өгөгдлийн сан үүсгэх SQL код 19
A.2 Байршил илрүүлэх алгоритмын Python код 20
A.3 dashboard-д шаардлагатай өгөгдлийг нэгтгэх ViewModel 26
A.4 Controllers-Үндсэн логикын жишээ 27

УДИРТГАЛ

Энэхүү тайланд UNIT LLC – Юу Эн Ай Ти XXK байгууллагад 2026 оны 01 сарын 08-ны өдрөөс 01 сарын 29-ний өдрийг хүртэл хугацаанд хийсэн үйлдвэрлэлийн дадлагын ажлын хүрээнд гүйцэтгэсэн ажил, судалгаа, хэрэгжүүлэлт болон үр дүнг нэгтгэн тайлагнав.

Дадлагын зорилго нь бодит програм хангамжийн бүтээгдэхүүн (sumbee.mn) дээр ажиллах замаар:

- Байгууллагын хөгжүүлэлтийн орчин, ашигладаг технологи, багийн ажлын зохион байгуулалттай танилцах
- Хөгжүүлж буй үндсэн системийн болон түүний үйл ажиллагааны ойлголт авах,
- Удирдагчийн зааварчилгааны дагуу алхам алхмаар судалгаа хийж өгсөн шаардлагын хүрээнд хэрэгжүүлэлт хийх явдал юм.

Дээрх зорилгын хүрээнд дадлагын хугацаанд python сангууд болон NER техникийг ашиглан монгол улсын аймаг, хот, сум, дүүрэг, болон түгээмэл ашиглагддаг байршлын нэршлүүд бүхий dictionary үүсгэж, тухайн dictionary-г ашиглан fb-ын постоос байршлыг ялгаж авдаг алгоритм бичсэн. Боловсруулсан датаг MSSQL-д бүтэцлэн хадгалж ASP.NET MVC болон DataTables ашиглан датаг хэрэглэгчид ойлгомжтой хэлбэрээр (Dashboard) харуулж, засварлаж, өөрчлөх боломж бүхий page хуудас хөгжүүлсэн.

1. БАЙГУУЛЛАГЫН ТАНИЛЦУУЛГА

1.1 UNIT LLC/Юу Эн Ай Ти ХХК танилцуулга

UNIT LLC– Юу Эн Ай Ти ХХК нь 2015 онд байгуулагдсан их өгөгдөл, хиймэл оюун ухаанд суурилсан сошиал мониторинг систем хөгжүүлдэг мэдээллийн технологийн компани юм. Одоогоор уг компани нь Sumbee.mn гэх систем дээр голчлон ажиллаж байна.

1.2 Sumbee.mn системийн зорилго

Sumbee платформ нь facebook, twitter, instagram, youtube, threads зэрэг цахим сүлжээний платформууд болон Монголын идэвхтэй үйл ажиллагаа явуулдаг 160 гаруй мэдээний сайтын мэдээллийн урсгалыг нэгтгэсэн анализ, тайланг нэг платформоос авах боломжийг олгодог.

1.2.1 Social monitoring гэж юу вэ?

Сошиал медиа мониторинг систем нь сошиал орчин дахь хэрэглэгчдийн пост, жиргээ, сэтгэгдэл зэргийг цаг алдалгүй нэг платформд цуглуулж, ойлгомжтой хэлбэрээр харуулдаг систем юм. Энэ нь тухайн сэдвийн хэрэглэгчийн байр суурийг тодорхойлох, талуудын оролцоо, чиг хандлагыг хянах, нөлөөлөгчдийг илрүүлэх, зохион байгуулалттай үйлдэл илрүүлэх боломжийг олгодог.

1.2.2 Sumbee-ийн үндсэн боломжууд

- Брэндийн нэрээр хайлт хийх (keyword-based monitoring)
- Дурдагдлын тоон мэдээлэл (mention count)
- Эерэг/сөрөг/саармаг ангилалт (sentiment analysis)
- Тайлан, график, дашбоард
- Шуурхай мэдэгдэл (alert / notification)

2. ИЖИЛ СИСТЕМҮҮДИЙН СУДАЛГАА

2.1 Олон улсад

Social monitoring болон social listening чиглэлээр дэлхийд өргөн хэрэглэгддэг зарим системүүд:

- Brandwatch
- Sprout Social
- Meltwater
- Hootsuite Insights

Эдгээр системүүд нь олон улсын зах зээлд чиглэсэн, их хэмжээний өгөгдөл боловсруулдаг бөгөөд sentiment analysis, trend detection, influencer analysis зэрэг дэвшилтэт боломжуудыг агуулдаг.

2.2 Монголын зах зээл дэх мониторингийн хэрэгслүүд

Монголын зах зээлд сошиал мониторингийг дараах байдлаар гүйцэтгэх нь түгээмэл байдаг:

- Facebook Page Insights
- Гар аргаар тайлан бэлтгэх
- PR агентлагийн судалгаа, тайлан

Гэсэн хэдий ч эдгээр аргууд нь автоматжуулалт багатай, бодит хугацааны мониторинг хийхэд хүндрэлтэй байдаг.

2.3 Sumbee.mn системийн давуу тал

Sumbee.mn нь Монгол хэл дээрх пост, текстийн онцлогт тохируулсан мониторингийн систем бөгөөд:

- Дотоодын мэдээний сайтуудыг, нийгмийн сүлжээг хамарсан өгөгдөл ашигладаг
- Түлхүүр үгт суурилсан их өгөгдөлд хиймэл оюун ухаанаар анализ хийдэг
- Өгөгдлийн нэг дор real-time цуглуулж, боловсруулан, шинжилгээ хийж хэрэглэгчид ойлгомжтой, энгийн байдлаар харуулах цогц үйл үйлчилгээг үзүүлдэг

зэрэг давуу талуудтай.

3. СИСТЕМИЙН ШААРДЛАГА

Энэхүү бүлэгт Sumbee.mn сошиал мониторинг системийн хүрээнд дадлагын хугацаанд хэрэгжүүлсэн “постын текстээс байршлын мэдээлэл ялган авах” feature-ийн системийн шаардлагыг тодорхойлно.

3.1 Системийн хэрэглэгч

Уг feature-ийн үндсэн хэрэглэгч нь сошиал мониторингийн үр дүнг ашиглан дүн шинжилгээ хийдэг байгууллагын маркетинг, PR болон хэрэглэгчийн дэмжлэгийн багийн ажилтан байна. Хэрэглэгч нь постууд аль байршилтай холбоотой байгааг харах, байршлаар нь ангилсан мэдээллийг ашиглан шийдвэр гаргах зорилготой.

3.2 Функциональ хэрэглэгчийн шаардлага

Постын текстээс байршлын мэдээлэл ялган авах feature нь дараах функциональ шаардлагуудыг хангана:

- Сошиал медиагаас цугларсан постын текстийг унших
- Постын текст доторх байршлын нэршлийг автоматаар илрүүлэх
- Илэрсэн байршлын мэдээллийг тухайн посттой холбон хадгалах
- Байршлын мэдээлэл илэрсэн постуудыг жагсаалтаар харах
- Байршлаар ангилсан постын болон илэрсэн байршлын тоон мэдээлэл, статистик харах(нийт постын тоо, байршил илэрсэн постын тоо, хамгийн их давтамжтай байршилууд зэрэг мэдээллүүд байна.)
- Газрын зураг дээр байршлуудыг илэрцийн давтамжаар ялгаж харуулах
- Байршлын нэрээр хайлт хийх боломжтой байх
- Fb account-аар хайлт хийх боломжтой байх
- Илэрсэн байршил, постын хамаарлын бүртгэлийг засаж, өөрчлөх боломжтой байх

3.3 Функциональ бус шаардлага

Уг feature нь дараах функциональ бус шаардлагуудыг хангах ёстой:

- Байршлын мэдээлэл ялгах процесс нь мониторингийн үндсэн урсгалыг удаашруулахгүй байх
- Байршлын өгөгдөл нь алдаагүй, давхардалгүй хадгалагдах
- Шинэ байршлын нэршил нэмэх боломжтой байхаар зохиогдсон байх
- Өгөгдөл боловсруулах логик нь ойлгомжтой, засварлахад хялбар байх

4. АШИГЛАСАН ТЕХНОЛОГИУД

Энэхүү бүлэгт Sumbee.mn сошиал мониторинг системийн хүрээнд дадлагын хугацаанд хэрэгжүүлсэн “постын текстээс байршлын мэдээлэл ялган авах болон dashboard-д харуулах” feature-д ашигласан технологиудыг тайлбарлана.

4.1 Python програмчлалын хэл

Python нь текст боловсруулалт, өгөгдөл боловсруулах чиглэлээр өргөн хэрэглэгддэг, өндөр түвшний програмчлалын хэл юм.Дадлагын хүрээнд Python програмчлалын хэлийг постын текстээс байршлын мэдээлэл ялган авах алгоритм хэрэгжүүлэхэд ашигласан. Python нь:

- Текст боловсруулалтад зориулсан олон сан (regex, string processing)
- Код уншихад ойлгомжтой, хурдан хөгжүүлэлт хийх боломж
- NLP чиглэлд өргөн хэрэглэгддэг

давуу талуудтай тул уг feature-д тохиромжтой гэж үзсэн.

4.1.1 Feature-д гүйцэтгэсэн үүрэг

Python ашиглан:

- Постын текстийг урьдчилан цэвэрлэх
- Dictionary-д суурилсан байршил илрүүлэх
- Илэрсэн байршлын үр дүнг бүтэцлэн боловсруулах

ажлуудыг гүйцэтгэсэн.

4.2 Dictionary-based approach

Dictionary-based approach нь урьдчилан тодорхойлсон нэршлийн жагсаалт ашиглан текстээс утга илрүүлэх аргачлал юм. Энэхүү аргачлалыг ашиглан:

Монгол Улсын:

- Аймаг
- Хот
- Сум
- Дүүрэг

болон түгээмэл байршлын нэршлүүдийг агуулсан dictionary үүсгэн, постын текстээ харьцуулан байршлыг илрүүлсэн.

4.2.1 Давуу тал

- Сургалтын өгөгдөл шаарддаггүй
- Үр дүнг тайлбарлах боломжтой
- Бодит системд хурдан нэвтрүүлэх боломжтой

4.3 MSSQL өгөгдлийн сан

Илэрсэн байршлын мэдээлэл болон постын өгөгдлийг хадгалах зорилгоор Microsoft SQL Server (MSSQL) өгөгдлийн сан ашигласан.

4.3.1 Яагаад MSSQL сонгосон бэ?

- Бүтэцлэгдсэн өгөгдөл хадгалахад тохиромжтой
- ASP.NET технологитой нягт уялддаг
- Query бичих, тайлан гаргахад хялбар

4.3.2 Feature-д гүйцэтгэсэн үүрэг

- Үүсгэсэн dictionary өгөгдлөө өгөгдлийн санд хадгалаад өгөгдлийн сангаас хайлт хийдэг болгох
- Пост болон байршлын мэдээллийг холбоотой хадгалах
- Байршлаар ангилсан өгөгдөл гаргах
- Dashboard-д ашиглагдах query-г дэмжих

4.4 Backend хөгжүүлэлт – ASP.NET MVC

ASP.NET MVC нь Model-View-Controller архитектурт суурилсан веб хөгжүүлэлтийн фрэймворк бөгөөд өгөгдөл боловсруулах логик, хэрэглэгчид харагдах интерфэйс, хүсэлт хариу боловсруулах процессийг тусгаарлах боломжийг олгодог.

4.4.1 Архитектурын зохион байгуулалт

Дадлагын хүрээнд хэрэгжүүлсэн байршил ялгах feature-ийн backend архитектур нь дараах бүрэлдэхүүн хэсгүүдээс бүрдэнэ:

- **Controller** – Хэрэглэгчийн хүсэлтийг хүлээн авч, тохирох өгөгдлийг бэлтгэх, байршлын өгөгдлийг боловсруулах логик
- **Model** – Өгөгдлийн сангийн бүтэцтэй уялдсан өгөгдлийн загвар

4.5 Frontend хөгжүүлэлт – ASP.NET MVC

Frontend хэсэг нь ASP.NET MVC-ийн View давхаргад суурилсан бөгөөд Razor template engine ашиглан backend-с ирсэн өгөгдлийг хэрэглэгчид ойлгомжтой байдлаар харуулдаг.

4.5.1 Razor View ба өгөгдөл дамжуулалт

Razor View нь C# болон HTML-г хослуулан ашиглах боломж олгодог. Controller-оос дамжуулсан model өгөгдлийг:

- Хүснэгт
- Статистик блок
- Тоон үзүүлэлт

хэлбэрээр дүрсэлсэн.

4.5.2 DataTables ашигласан интерфэйс

Байршлын өгөгдөлтэй постуудыг харуулахдаа DataTables JavaScript санг ашигласан. Уг сан нь:

- Хайлт
- Байршлаар шүүлт хийх
- Pagination

боломжийг хэрэглэгчид олгодог.

4.5.3 JavaScript ба UI логик

Frontend талд JavaScript ашиглан:

- Хугацаа болон байршлын filter
- Хүснэгтийн динамик шинэчлэлт
- Хэрэглэгчийн үйлдэлд хариу өгөх

логик хэрэгжүүлсэн.

5. ХЭРЭГЖҮҮЛЭЛТ

Энэхүү бүлэгт Sumbee.mn сошиал мониторинг системийн хүрээнд дадлагын хугацаанд хэрэгжүүлсэн “постын текстээс байршлын мэдээлэл ялган авах” feature-ийн хэрэгжилтийг алхамчилсан байдлаар тайлбарлана.

5.1 Байршлын dictionary үүсгэх

Байршлын мэдээлэл илрүүлэх зорилгоор Монгол Улсын газарзүйн нэршлүүдийг агуулсан dictionary өгөгдөл үүсгэсэн. Dictionary -д ашиглах өгөгдлийг Overpass API ашиглан OpenStreetMap - ийн нээлттэй эхийн өгөгдлөөс цуглуулсан.Overpass API нь OpenStreetMap-ийн асар том өгөгдлийн сангаас зөвхөн өөрт хэрэгцээтэй хэсгийг (жишээ нь: Монгол улс дахь бүх дүүрэг, хороо) шүүж авах боломжийг олгодог хүчирхэг хайлтын хэрэгсэл юм. Өгөгдлүүдийг дараах бүтэцтэйгээр цуглуулж, fb постын өгөгдөл дээр давхар шүүлт хийж хайлтыг илүү оновчтой болгох үүднээс түгээмэл гарч ирж буй байршлуудын нэршлүүдийг үлдээж, ашиглагдахгүй байгаа байршлуудын нэршлүүдийг хасаж dictionary-г илүү оновчтой болгосон.

- Аймаг, нийслэл
- Сум, дүүрэг
- Түгээмэл хэрэглэгддэг товчилсон хэлбэрүүд/автобусны буудлуудыг нэршил/

5.2 Байршлын мэдээлэл ялгах алгоритм

Цэвэрлэсэн постын текстийг dictionary өгөгдөлтэй харьцуулан байршлын нэршлийг автоматаар илрүүлэх логик хэрэгжүүлсэн.Python pandas, re , json, requests зэрэг санг ашиглан token-д суурилсан, context-aware логик бүхий алгоритм бичсэн.

5.2.1 Байршлын нэршил илрүүлэх алгоритмын ажиллах зарчим

Алгоритмын эхний шатанд постын текстийг normalize хийж, байршил илрүүлэхэд тохиромжтой хэлбэрт оруулна.

- Бүх үсгийг жижиг үсэгт шилжүүлэх

- Тусгай тэмдэг, цэг, таслал, emoji-г арилгах
- Илүүдэл хоосон зайг цэвэрлэх

Normalize хийсэн текстийг space-аар салган token (үгс)-д хуваана. Token-д суурилсан арга нь олон үгтэй байршлын нэршлийг зөв дарааллаар илрүүлэх суурь болдог. Байршлын dictionary нь canonical нэршил болон түүний alias хэлбэрүүдээс бүрдэнэ.

- Canonical name – албан ёсны байршлын нэр
- Alias – тухайн байршлын ярианы, товчилсон, хувилбар бичлэг

Алгоритм нь alias-уудыг эхний үгээр нь индексжүүлсэн (alias index) бүтэц ашиглан хайлтыг оновчтой болгосон.

- Token нь alias-ийн эхний үгтэй таарах эсэхийг шалгаж, таарч байвал үргэлжлүүлэн шалгаж, таарахгүй байвал тухайн index ийг алгасаж дараагийн индексийг шалгах замаар хайлтыг оновчтой болгосон.
- Тухайн index дотроо хайлт хийхдээ олон үгтэй alias-уудыг уртаар нь эрэмбэлж, амгийн уртаас нь эхэлж match хийснээр илүү оновчтой олон үг нийлсэн байрлалыг ч ялгаж таньдаг болсон.

Алгоритм нь token бүр дээр давталт хийж, дээрх аргаар шалган таарч буй alias уудыг candidate болгон сонгоно. Candidate alias бүр дээр дараах шалгалтыг хийдэг:

- Alias-ийн бүх token нь текстийн дарааллалтай таарч байгаа эсэх
- Alias-ийн сүүлийн үг нь suffix-тэй нийлж бичигдсэн эсэх
- Байршлын дараа “д”, “руу”, “дээр”, “орчим” гэх мэт үг орсон эсэх зэргийг давхар шалгана.

Match баталгаажсан тохиолдолд:

- Canonical ID
- Alias ID

өгөгдлийг тухайн посттой холбон өгөгдлийн санд хадгална.

5.2.2 Алгоритмын онцлог

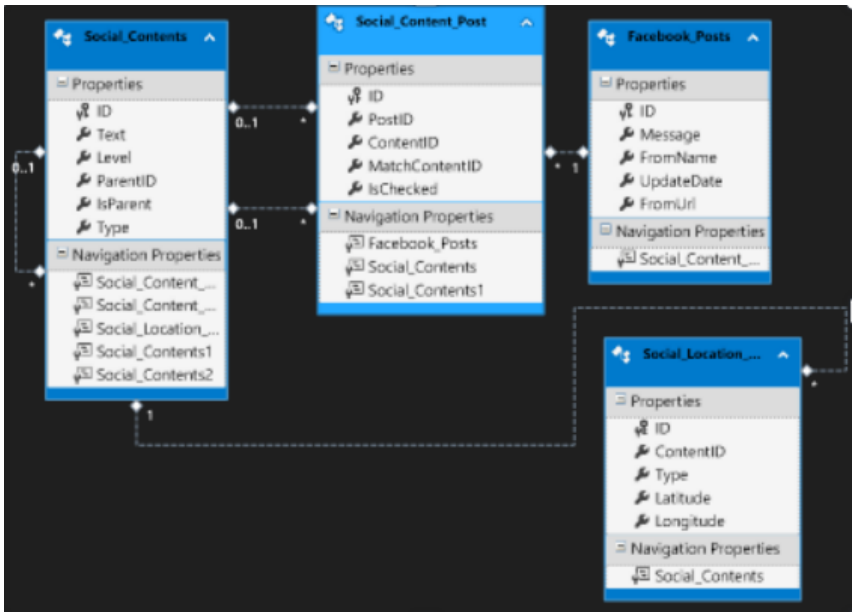
- Монгол хэлний ярианы хэллэг, товчлолыг дэмждэг
- Dictionary-д суурилсан тул үр дүн тайлбарлах боломжтой.
- Context-aware логик ашигласан нь буруу илрүүлэлтийг бууруулдаг
- Бодит системд хурдан, тогтвортой ажилладаг

5.3 Өгөгдлийн санд хадгалах

Илэрсэн байршлын мэдээллийг MSSQL өгөгдлийн санд бүтэцлэн хадгалсан.

- Dictionary дахь байршлуудыг 1-6 хүртэлх түвшинд ангилан мод бүтцээр хадгалсан.
- Байршилын үндсэн нэршил/canonical idea/, alias/ярианы хэллэг, товчлол/, тэдгээрийн уртраг, өргөрөг зэрэг мэдээллүүдийг нэгтгэн хадгалсан.
- Пост болон түүн дээр илэрсэн байршлын хамаарлыг нэг table болгон хадгалсан

Өгөгдлийг тусгаарласан бүтэцтэй хадгалснаар байршлаар шүүх, статистик гаргах боломж бүрдсэн.



Зураг 5.1: Өгөгдлийн сангийн бүтэц

5.4 Хөгжүүлэлтийн орчин тохируулах

Хөгжүүлэлтийг эхлүүлэхийн өмнө шаардлагатай програм хангамж болон орчны тохиргоог хийв.

- Visual Studio 2022 – ASP.NET MVC төслийг хөгжүүлэх
- TortoiseSVN – Төслийн эх кодыг татаж авах, хувилбарын удирдлага
- Microsoft SQL Server – өгөгдлийн сан

5.4.1 Хөгжүүлэлтийн орчин суулгах

Visual Studio 2022 программыг суулгаж, ASP.NET болон веб хөгжүүлэлтийн workload-уудыг сонгон идэвхжүүлсэн.

5.4.2 Төслийн эх код татах

TortoiseSVN ашиглан төслийн эх кодыг репозиториос local орчинд checkout хийж татсан.

5.4.3 Dependencies ба өгөгдлийн сангийн холболт

Төслийг татаж авсны дараа:

- NuGet packages сэргээх
- MSSQL өгөгдлийн сантай холболтыг тохируулах

5.4.4 Төслийг local орчинд ажиллуулах

Төслийг Visual Studio-ooc build хийж, local орчинд амжилттай ажиллуулж туршиж өгөгдлийн сангийн холболт болон бусад тохиргоонууд зөв ажиллаж байгаа эсэхийг баталгаажуулсан.

5.5 Вэб хөгжүүлэлт

Байршлын мониторингийн feature-ийн хүрээнд ASP.NET MVC framework ашиглан backend болон frontend логикийг хэрэгжүүлсэн.

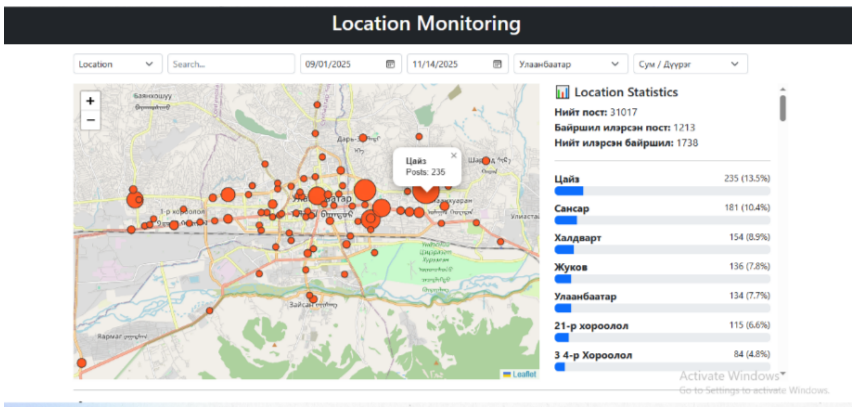
5.5.1 Backend логик (ASP.NET MVC)

Backend хэсэгт MVC архитектурыг баримтлан өгөгдөл боловсруулах логикийг хэрэгжүүлсэн.

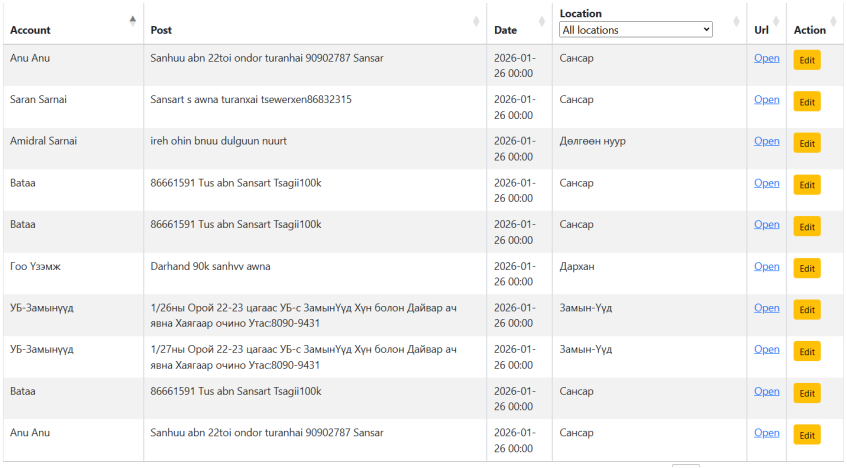
- Controller – Хэрэглэгчийн хүсэлтийг хүлээн авч, байршлын өгөгдлийг өгөгдлийн сангаас татан бэлтгэх, байршлын өгөгдөл нэгтгэх, шүүх бизнес логик
- Model – MSSQL өгөгдлийн сантай харилцах өгөгдлийн загвар

Frontend хэсэгт ASP.NET MVC-ийн Razor View болон JavaScript суурьтай DataTables ашиглан байршлын мониторингийн үр дүнг дүрслэв.

- Байршлаар ангилсан постын болон илэрсэн байршлын тоон мэдээлэл, статистик
- Байршлын мэдээлэл илэрсэн постуудыг жагсаалт
- Газрын зураг дээр байршлуудыг илэрцийн давтамжаар ялгаж харуулах
- Байршлын нэрээр хайлт хийх
- Хугацааны шүүлтүүр
- Бүс нутгын шүүлтүүр
- Илэрсэн байршил, постын хамаарлын бүртгэлийг засаж, өөрчлөх боломжтой цонх

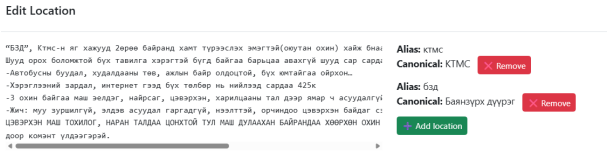


Зураг 5.2: Газрын зураг,статистик бүхий dashboard



Account	Post	Date	Location (All locations)	Url	Action
Anu Anu	Sanhuu abn 22toi ondor turanhai 90902787 Sansar	2026-01-26 00:00	Сансар	Open	Edit
Saran Sarnai	Sansart s awna turanxai tsewerxen86832315	2026-01-26 00:00	Сансар	Open	Edit
Amidral Sarnai	ireh ohin bnuu dulguun nuurt	2026-01-26 00:00	Дөлгөөн нуур	Open	Edit
Bataa	86661591 Tus abn Sansart Tsagii100k	2026-01-26 00:00	Сансар	Open	Edit
Bataa	86661591 Tus abn Sansart Tsagii100k	2026-01-26 00:00	Сансар	Open	Edit
Гоо Үзэмж	Darhand 90k sanhvv awna	2026-01-26 00:00	Дархан	Open	Edit
УБ-Замынүүд	1/26ны Орой 22-23 цагаас УБ-с ЗамынҮүд Хүн болон Дайвар ач явна Хаягаар очино Утас:8090-9431	2026-01-26 00:00	Замын-Үүд	Open	Edit
УБ-Замынүүд	1/27ны Орой 22-23 цагаас УБ-с ЗамынҮүд Хүн болон Дайвар ач явна Хаягаар очино Утас:8090-9431	2026-01-26 00:00	Замын-Үүд	Open	Edit
Bataa	86661591 Tus abn Sansart Tsagii100k	2026-01-26 00:00	Сансар	Open	Edit
Anu Anu	Sanhuu abn 22toi ondor turanhai 90902787 Sansar	2026-01-26 00:00	Сансар	Open	Edit

Зураг 5.3: Байршлын мэдээлэл илэрсэн постуудын жагсаалт



Зураг 5.4: Бүртгэлийг засаж, өөрчлөх боломжтой цонх

5.6 Туршилт ба баталгаажуулалт

Хэрэгжүүлсэн feature-ийн ажиллагааг баталгаажуулахын тулд гараар болон системийн түвшний шалгалтууд хийсэн.

- Байршил илрүүлэх логикийн үр дүнг гараар шалгах
- Dashboard дээр зөв харагдаж байгаа эсэхийг тестлэх
- Алдаатай болон хоосон өгөгдөл оруулах үед системийн тогтвортой байдлыг шалгах

ДҮГНЭЛТ

UNIT LLC – Юу Эн Ай Ти ХХК байгууллагад хийсэн үйлдвэрлэлийн дадлагын хугацаанд Sumbee.mn сошиал мониторинг системийн хүрээнд бодит өгөгдөл дээр ажиллаж, постын текстээс байршлын мэдээлэл ялган авах feature-ийг амжилттай хэрэгжүүллээ.

Дадлагын явцад сошиал медиа постуудын бүтэцгүй текстэн өгөгдлийн онцлог, Монгол хэлний ярианы хэллэг, товчилсон болон олон хувилбартай бичлэгүүд нь өгөгдөл боловсруулахад тодорхой хүндрэл үүсгэдгийг практик дээрээс ойлгосон. Эдгээр хүндрэлүүдийг харгалзан dictionary-based болон context-aware логикт суурилсан алгоритм боловсруулж, байршлын нэршлийг найдвартай илрүүлэх шийдэл хэрэгжүүлсэн.

Хэрэгжүүлсэн алгоритм нь цэвэрлэсэн текстийг alias dictionary өгөгдөлтэй харьцуулж, байршлын төрөл, suffix болон өгүүлбэрийн орчныг харгалзан шийдвэр гаргадаг бөгөөд бодит мониторингийн системд тогтвортой ажиллах боломжтойг туршилтаар баталгаажуулсан. Илэрсэн байршлын өгөгдлийг MSSQL өгөгдлийн санд бүтэцлэн хадгалж, ASP.NET MVC технологи ашиглан dashboard хэлбэрээр хэрэглэгчид ойлгомжтой байдлаар харуулсан нь системийн бодит хэрэглээнд ач холбогдолтой шийдэл болсон.

Энэхүү дадлага нь зөвхөн програмчлалын ур чадварыг нэмэгдүүлэхээс гадна бодит системийн архитектур, өгөгдлийн урсгал, бизнес логик болон хэрэглэгчийн шаардлагыг уялдуулан ажиллах чухал туршлага болж өгсөн.

Цаашид энэхүү дадлагаас олж авсан туршлагад тулгуурлан байршлын илрүүлэлтийн нарийвчлалыг нэмэгдүүлэх зорилгоор machine learning болон NLP аргачлалуудыг шат дараатайгаар нэвтрүүлэх, dictionary-based болон AI-д суурилсан hybrid шийдэл боловсруулах,мөн мониторингийн өгөгдөл дээр илүү гүнзгий аналитик хийх чадвараа цаашид хөгжүүлэх зорилготой байна.

НОМ ЗҮЙ

[1] Microsoft. *ASP.NET MVC Documentation*. [Online]. Available: `https://learn.microsoft.com/aspnet/mvc`.

[2] Microsoft. *SQL Server Documentation*. [Online]. Available: `https://learn.microsoft.com/sql`.

[3] Python Software Foundation. *Text Processing Services*. [Online]. Available: `https://docs.python.org/3/library/text.html`.

[4] TortoiseSVN Project. *TortoiseSVN Documentation*. [Online]. Available: `https://tortoisesvn.net/docs.html`.

[5] Git Project. *Git Version Control Documentation*. [Online]. Available: `https://git-scm.com/doc`.

[6] Jurafsky, D., & Martin, J. H. *Speech and Language Processing: An Introduction to Natural Language Processing*. Prentice Hall, 2023. (Dictionary-based approach болон NER судалгааны хүрээнд).

[7] Bird, S., Klein, E., & Loper, E. *Natural Language Processing with Python*. O'Reilly Media, 2009. (Текст боловсруулалтын аргачлалын хүрээнд).

[8] Сошиал медиа мониторингийн ерөнхий ойлголт, чиг хандлага. Судалгааны тайлан, 2024.

A. ХАВСРАЛТ Кодын хэсэг

Өгөгдлийн сан үүсгэх SQL код:

```
1
2  CREATE DATABASE SocialDataLocal;
3  USE SocialDataLocal;
4  -- 1. Facebook Posts table
5  CREATE TABLE Facebook_Posts (
6      ID UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
7      Message NVARCHAR(MAX),
8      FromName NVARCHAR(255),
9      UpdateDate DATETIME,
10     FromUrl NVARCHAR(MAX)
11 );
12 -- 2. Social Contents
13 CREATE TABLE Social_Contents (
14     ID UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
15     Text NVARCHAR(500) NOT NULL,
16     Level INT,
17     ParentID UNIQUEIDENTIFIER NULL,
18     IsParent BIT DEFAULT 0,
19     Type NVARCHAR(50),
20     CONSTRAINT FK_SocialContents_Parent FOREIGN KEY (ParentID)
          REFERENCES Social_Contents(ID)
21 );
22 -- 3. Social Location More
23 CREATE TABLE Social_Location_More (
24     ID UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
25     ContentID UNIQUEIDENTIFIER NOT NULL,
26     Type NVARCHAR(50),
27     Latitude NVARCHAR(50),
28     Longitude NVARCHAR(50),
29     CONSTRAINT FK_LocationMore_Content FOREIGN KEY (ContentID)
          REFERENCES Social_Contents(ID) ON DELETE CASCADE
30 );
31 -- 4. Social Content Post (Matching table)
32 CREATE TABLE Social_Content_Post (
33     ID UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
34     PostID UNIQUEIDENTIFIER NOT NULL,
```

```
    ContentID UNIQUEIDENTIFIER NULL,
    MatchContentID UNIQUEIDENTIFIER NULL,
    IsChecked BIT DEFAULT 0,
    -- Foreign Keys
    CONSTRAINT FK_ContentPost_Post FOREIGN KEY (PostID) REFERENCES
       Facebook_Posts(ID) ON DELETE CASCADE,
    CONSTRAINT FK_ContentPost_Content FOREIGN KEY (ContentID)
       REFERENCES Social_Contents(ID),
    CONSTRAINT FK_ContentPost_MatchContent FOREIGN KEY (
       MatchContentID) REFERENCES Social_Contents(ID)
);
```

Код А.1: Өгөгдлийн сан үүсгэх SQL код

Байршил илрүүлэх алгоритмын Python код:

```
import pyodbc
import re
from collections import defaultdict
import uuid

#server_name = '10.10.15.202'
#database_name = 'sumbee'
#username = "user_sumbee"
#password = "225RQ7_sH)Lb"

#conn_str = (
#    "DRIVER={ODBC Driver 17 for SQL Server};"
#    f"SERVER={server_name};"
#    f"DATABASE={database_name};"
#    f"UID={username};"
#    f"PWD={password}"
#)
conn_str = (
    "DRIVER={ODBC Driver 17 for SQL Server};"
    "SERVER=localhost\\SQLEXPRESS;"
    "DATABASE=SocialDataLocal;"
    "Trusted_Connection=yes;"
    "TrustServerCertificate=yes;"
)
conn = pyodbc.connect(conn_str)
cursor = conn.cursor()
conn.autocommit = False
```

```
28
29  context_words = {
30      "  ","  ","      ","      ","        ","          ","
        ",
31      "d","t","s","ruu","orchim","hawiar","habiar",
32      "hawid","havitsaa","havit","havid","haviar","havair",
33      "der","deer","giin","iin","        ","        ","yvna","yvah",
34      "yumu","ymu","    ","    ","eswel","esvel","          ",
35      "horoolol","horooll","               ","xorooll","xoroolol",
36      "r","  ","dugeer","dvgeer","myangatad"
37  }
38
39  def normalize(text):
40      if not text:
41          return ""
42      text = text.lower()
43      text = re.sub(r"[^\w\s]", " ", text)
44      return re.sub(r"\s+", " ", text).strip()
45
46  def check_suffix_context(token, alias_base):
47      #13rhoroolol, 120t, 5ruu                suffix
48      if token.startswith(alias_base) and len(token) > len(alias_base
        ):
49          suffix = token[len(alias_base):]
50          return suffix in context_words
51      return False
52
53  def check_common_logic(tokens, current_idx, match_len, alias_tokens
    ):
54      #COMMON                                  context-aware
55      n = len(tokens)
56      last_idx = current_idx + match_len - 1
57
58      last_text_token = tokens[last_idx]
59      last_alias_part = alias_tokens[-1]
60
61      # CASE 1: suffix              context
62      if check_suffix_context(last_text_token, last_alias_part):
63          return True, 0
```

```
64
65      # CASE 2:                               context
66      next_ptr = current_idx + match_len
67      if next_ptr < n:
68          next_token = tokens[next_ptr]
69
70          if next_token in context_words:
71              return True, 1
72
73          #
74          if next_token in alias_index: #if next_token in alias_index
                .keys():
75              return True, 0
76      return False, 0
77
78  def load_alias_index():
79      #Level 6 alias + canonical + location type
80      cursor.execute("""
81          SELECT
82              a.ID            AS AliasID,
83              a.Text          AS AliasText,
84              c.ID            AS CanonicalID,
85              c.Text          AS CanonicalText,
86              loc.Type        AS LocationType
87          FROM [sumbee].[dbo].[Social.Contents] as a
88          JOIN [sumbee].[dbo].[Social.Contents] as c ON a.ParentID =
                c.ID
89          JOIN [sumbee].[dbo].[Social.Location.More] as loc ON loc.
                ContentID = c.ID
90          WHERE a.Level = 6
91      """)
92      optimized_index = {}
93
94      def add_to_index(row):
95          tokens = row.AliasText.lower().strip().split()
96          if not tokens:
97              return
98
99          first_word = tokens[0]
100         optimized_index.setdefault(first_word, [])
```

```
101
102         if not any(
103             x["full_alias_tokens"] == tokens and x["canonical_id"]
                    == row.CanonicalID
104             for x in optimized_index[first_word]
105         ):
106             optimized_index[first_word].append({
107                 "full_alias_tokens": tokens,
108                 "alias_id": row.AliasID,
109                 "canonical_id": row.CanonicalID,
110                 "location_type": row.LocationType,
111                 "length": len(tokens)
112             })
113
114     for row in cursor.fetchall():
115         print("fetching aliases")
116         add_to_index(row)
117     return optimized_index
118
119 alias_index = load_alias_index()
120 # ================= MATCH LOGIC ========================
121
122 def match_locations(text):
123     tokens = normalize(text).split()
124     if not tokens:
125         return []
126     found = []
127     i = 0
128     n = len(tokens)
129     while i < n:
130         token = tokens[i]
131         candidates = []
132         for k in alias_index:
133             if token.startswith(k):
134                 candidates.extend(alias_index[k])
135         candidates.sort(key=lambda x: x["length"], reverse=True)
136         matched = False
137         for item in candidates:
138             alias_tokens = item["full_alias_tokens"]
139             m = len(alias_tokens)
140             if i + m > n:
```

```
                continue
            if any(tokens[i + j] != alias_tokens[j] for j in range(
                m - 1)):
                continue
            last_text_token = tokens[i + m - 1]
            last_alias_part = alias_tokens[-1]
            if not last_text_token.startswith(last_alias_part):
                continue
            if item["location_type"] == "common":
                ok, skip = check_common_logic(tokens, i, m,
                    alias_tokens)
                if ok:
                    found.append({
                        "canonical_id": item["canonical_id"],
                        "alias_id": item["alias_id"]
                    })
                    i += m + skip
                    matched = True
                    break
            else:
                if len(last_text_token) <= len(last_alias_part) +
                    4:
                    found.append({
                        "canonical_id": item["canonical_id"],
                        "alias_id": item["alias_id"]
                    })
                    i += m
                    matched = True
                    break

        if not matched:
            i += 1
    return found
# ================ LOAD POSTS ========================
def load_posts():
    cursor.execute("""
        SELECT p.ID, p.Message
        FROM [sumbee].[dbo].[Facebook.Posts] as p
        JOIN [Facebook.Post.Person] as per ON p.ID = per.PostID
        WHERE per.PersonID = ?
          AND p.UpdateTime > ?
```

```
        ORDER BY p.UpdateTime DESC
    """, '7c354d80-70e5-419f-b36c-193e0f4601fb', '2026-01-16')

    return cursor.fetchall()
# ================= INSERT RESULT ======================

def insert_content_post(p_id, c_id, a_id):
        cursor.execute("""
            IF NOT EXISTS (
                SELECT 1 FROM [sumbee].[dbo].[Social.Content.Post]
                WHERE PostID = ? AND ContentID = ? AND
                   MatchContentID = ?
            )
            INSERT INTO [sumbee].[dbo].[Social.Content.Post]
            (PostID, ContentID, MatchContentID, IsChecked)
            VALUES (?, ?, ?, ?)
        """, p_id, c_id, a_id, p_id, c_id, a_id, 0)
try:
    posts = load_posts()
    print("loaded posts:matching started")
    for post in posts:
        matches = match_locations(post.Message)
        if matches:
            for m in matches:
                insert_content_post(
                    uuid.UUID(post.ID),
                    uuid.UUID(m["canonical_id"]),
                    uuid.UUID(m["alias_id"])
                )
        else:
            insert_content_post(
                    uuid.UUID(post.ID),
                    None,
                    None
                )
    conn.commit()
    print(" Content.Post insert                    ")
except Exception as e:
    conn.rollback()
    print(" ERROR:", e)

```

```
finally:
    cursor.close()
    conn.close()
```

Код А.2: Байршил илрүүлэх алгоритмын Python код

dashboard-д шаардлагатай өгөгдлийг нэгтгэх ViewModel:

```
using System;
using System.Collections.Generic;

namespace SocialMonster.Models
    //               Dashboard-        ViewModel
    public class LocationDashboardViewModel
    {
        public int TotalPosts { get; set; }
        public int TotalPostsWLoc { get; set; }
        public int UniqueLocationsCount { get; set; }
        public List<TopLocationStat> TopLocations { get; set; }
    }
    public class TopLocationStat
    {
        public string Name { get; set; }
        public int Count { get; set; }
        public decimal Percentage { get; set; }
    }
    //                                        (DataTable-
                          )
    public class LocationListViewModel
    {
        public Guid PostID { get; set; }
        public string FromName { get; set; }
        public string PostContent { get; set; }
        public string UpdatedTime { get; set; }
        public string Url { get; set; }
        public string LocationName { get; set; } // Canonical Name
        public bool HasLocation { get; set; }
    }
    //                            (Modal) ViewModel
    public class LocationEditViewModel
    {
        public Guid PostId { get; set; }
        public string FullContent { get; set; }
```

```
35          public List<DetectedAliasViewModel> CurrentMappings { get;
                set; }
36      }
37      public class DetectedAliasViewModel
38      {
39          public Guid ContentPostID { get; set; } // Social.Content.
                Post-       ID
40          public string CanonicalName { get; set; }
41          public string AliasName { get; set; }
42      }
43  }
```

Код А.3: dashboard-д шаардлагатай өгөгдлийг нэгтгэх ViewModel

Controllers-Үндсэн логикын жишээ:

```
1   using SocialMonster.DAL;
2   using SocialMonster.Models;
3   using System;
4   using System.Collections.Generic;
5   using System.Data.Entity;
6   using System.Linq;
7   using System.Text.RegularExpressions;
8   using System.Web.Mvc;
9   namespace SocialMonster.Controllers
10  {
11      //[Authorize]
12      public class LocationController : Controller
13      {
14          //private SumbeeEntities db = new SumbeeEntities();
15          private SocialDataLocalEntities db = new
                SocialDataLocalEntities();
16          public ActionResult Index()
17          {
18              return View();
19          }
20
21          public ActionResult Map()
22          {
23              return View();
24          }
25          //post filter
26          private IQueryable<Facebook_Posts> ApplyPostFilters(
```

```
            LocationFilter f)
27      {
28          IQueryable<Facebook_Posts> query = db.Facebook_Posts.
                AsNoTracking();
29          // Account search
30          /*if (!string.IsNullOrWhiteSpace(f.Search) && f.
                Category == "account")
31          {
32              var accountIds = ResolveAccountCanonicalIds(f.
                    Search);
33
34              query = query.Where(p =>
35                  db.Social_Account_Post.Any(ap =>
36                      ap.PostID == p.ID &&
37                      accountIds.Contains(ap.AccountID)));
38          }*/
39          // ApplyLocationFilters
40          if (!string.IsNullOrWhiteSpace(f.Search) && f.Category
                == "account")
41          {
42              query = query.Where(cp =>
43                  cp.FromName.Contains(f.Search));
44          }
45
46          // Date range
47          if (f.StartDate.HasValue)
48              query = query.Where(p => p.UpdateDate >= f.
                    StartDate);
49
50          if (f.EndDate.HasValue)
51              query = query.Where(p => p.UpdateDate <= f.EndDate)
                    ;
52
53          // Location-based search
54          if (!string.IsNullOrWhiteSpace(f.Search) && f.Category
                == "location")
55          {
56              var canonicalIds = ResolveCanonicalIdsFromSearch(f.
                    Search);
57
58              query = query.Where(p =>
```

```
                    db.Social_Content_Post.Any(cp =>
                        cp.PostID == p.ID &&
                        cp.ContentID.HasValue &&
                        canonicalIds.Contains(cp.ContentID.Value)))
                            ;
            }
            return query;
        }
        //location filter
        private IQueryable<Social_Content_Post>
            ApplyLocationFilters(LocationFilter f)
        {
            var query = db.Social_Content_Post
                .AsNoTracking()
                .Where(cp => cp.ContentID != null);

            //  Account search
            if (!string.IsNullOrWhiteSpace(f.Search) && f.Category
                == "account")
            {
                query = query.Where(cp =>
                    cp.Facebook_Posts.FromName.Contains(f.Search));
            }
            //  Location search
            if (!string.IsNullOrWhiteSpace(f.Search) && f.Category
                == "location")
            {
                var canonicalIds = ResolveCanonicalIdsFromSearch(f.
                    Search);

                query = query.Where(cp =>
                    cp.ContentID.HasValue &&
                    canonicalIds.Contains(cp.ContentID.Value));
            }
            // Date range
            if (f.StartDate.HasValue)
                query = query.Where(cp => cp.Facebook_Posts.
                    UpdateDate >= f.StartDate);

            if (f.EndDate.HasValue)
                query = query.Where(cp => cp.Facebook_Posts.
```

```
                    UpdateDate <= f.EndDate);

            //  Location dropdown
            if (!string.IsNullOrWhiteSpace(f.Location))
                query = query.Where(cp => cp.Social_Contents.Text
                    == f.Location);

            //  District / Province
            if (f.DistrictId.HasValue || f.ProvinceId.HasValue)
            {
                var validIds = ResolveLocationTreeIds(f.ProvinceId,
                    f.DistrictId);
                query = query.Where(cp => validIds.Contains(cp.
                    ContentID.Value));
            }

            return query;
        }

        private IQueryable<Guid> ResolveLocationTreeIds(Guid?
            provinceId, Guid? districtId)
        {
            if (districtId.HasValue)
            {
                return db.Social_Contents
                    .Where(c =>
                        c.ID == districtId ||
                        c.ParentID == districtId ||
                        db.Social_Contents.Any(p => p.ParentID ==
                            districtId && c.ParentID == p.ID)
                    )
                    .Where(c => c.Level != 6)
                    .Select(c => c.ID);
            }

            if (provinceId.HasValue)
            {
                var l3 = db.Social_Contents.Where(x => x.ParentID
                    == provinceId && x.Level == 3);
                var l4 = db.Social_Contents.Where(x => l3.Select(a
                    => a.ID).Contains(x.ParentID.Value));
```

```
127                 var l5 = db.Social_Contents.Where(x => l4.Select(a
                        => a.ID).Contains(x.ParentID.Value));
128                 return db.Social_Contents
129                     .Where(x =>
130                         x.ID == provinceId ||
131                         l3.Select(a => a.ID).Contains(x.ID) ||
132                         l4.Select(a => a.ID).Contains(x.ID) ||
133                         l5.Select(a => a.ID).Contains(x.ID))
134                     .Select(x => x.ID);
135             }
136             return Enumerable.Empty<Guid>().AsQueryable();
137         }
138 
139         // 1. Statistic
140         [HttpGet]
141         public JsonResult GetLocationStatistics(LocationFilter
                filter)
142         {
143             var postQuery = ApplyPostFilters(filter);
144             var total = postQuery.Select(p => p.ID).Distinct().
                    Count();
145 
146             var query = ApplyLocationFilters(filter);
147 
148 
149             var stats = query
150                 .GroupBy(x => x.Social_Contents.Text)
151                 .Select(g => new
152                 {
153                     Name = g.Key,
154                     Count = g.Count()
155                 })
156                 .OrderByDescending(x => x.Count)
157                 .ToList();
158 
159             var totalHits = stats.Sum(x => x.Count);
160 
161             return Json(new
162             {
163                 totalPosts = total,
164                 totalPostsWLoc = query.Select(x => x.PostID).
```

```
                    Distinct().Count(),
                totalLocationHits = totalHits,
                stats = stats.Select(x => new
                {
                    x.Name,
                    x.Count,
                    Percentage = totalHits == 0 ? 0 : (decimal)x.
                        Count * 100 / totalHits
                })
            }, JsonRequestBehavior.AllowGet);
        }

        [HttpGet]
        public JsonResult GetLocationList(LocationFilter filter,
            int draw, int start, int length)
        {
            //  Base post filters (account, date, search)
            var postQuery = ApplyPostFilters(filter);

            //  Location-based filtering (province / district /
                dropdown)
            IQueryable<Guid> postIdsFromLocation = null;

            if (!string.IsNullOrWhiteSpace(filter.Location)
                || filter.ProvinceId.HasValue
                || filter.DistrictId.HasValue)
            {
                var locationQuery = ApplyLocationFilters(filter);

                postIdsFromLocation = locationQuery
                    .Select(cp => cp.PostID)
                    .Distinct();
            }

            if (postIdsFromLocation != null)
            {
                postQuery = postQuery.Where(p =>
                    postIdsFromLocation.Contains(p.ID));
            }

            //  OnlyWithLocation (LIST only)
```

```
            if (filter.OnlyWithLocation == true)
            {
                postQuery = postQuery.Where(p =>
                    db.Social_Content_Post.Any(cp =>
                        cp.PostID == p.ID &&
                        cp.ContentID != null));
            }

            var recordsTotal = postQuery.Count();

            //  Paging
            var posts = postQuery
                .OrderByDescending(p => p.UpdateDate)
                .Skip(start)
                .Take(length)
                .Select(p => new
                {
                    p.ID,
                    p.FromName,
                    p.Message,
                    p.UpdateDate,
                    p.FromUrl
                })
                .ToList();

            var postIds = posts.Select(p => p.ID).ToList();

            // Load location names in ONE query
            var locations = db.Social_Content_Post
                .AsNoTracking()
                .Where(cp =>
                    postIds.Contains(cp.PostID) &&
                    cp.ContentID != null)
                .GroupBy(cp => cp.PostID)
                .Select(g => new
                {
                    PostID = g.Key,
                    LocationName = g.Select(x => x.Social_Contents.
                        Text).FirstOrDefault()
                })
                .ToList();
```

```
241
242             var data = posts.Select(p => new LocationListViewModel
243             {
244                 PostID = p.ID,
245                 FromName = p.FromName,
246                 PostContent = p.Message,
247                 UpdatedTime = p.UpdateDate?.ToString("yyyy-MM-dd HH
                        :mm"),
248                 Url = p.FromUrl,
249                 LocationName = locations.FirstOrDefault(l => l.
                        PostID == p.ID)?.LocationName
250             });
251
252             return Json(new
253             {
254                 draw,
255                 recordsTotal,
256                 recordsFiltered = recordsTotal,
257                 data
258             }, JsonRequestBehavior.AllowGet);
259         }
260     }
261 }
```

Код А.4: Controllers-Үндсэн логикын жишээ